Project: Meredith March 1, 2026

Alexander Read

Introduction

The positive implicational fragment of the intuitionistic propositional calculus is a language $\mathcal{L}^{\to}$ whose formulae are defined by the following grammar. Let $\mathsf{PV} \equiv \{\ p_i \mid i \geq 0\ \}$, an infinite set of propositional variables, and let ϕ, ψ be meta-variables ranging over formulae:

$$\phi, \psi ::= \mathsf{PV} \mid \phi \to \psi$$

I've defined PV in this way because it closely mirrors my implementation in Haskell. However, in this post I'll use p, q, r as variables to match the usual notation.

Anyway, following Łukasiewicz, Meredith, Prior, and others (e.g., see here), a lot of the papers I've been reading on $\mathcal{L}^{\to}$ use prefix, rather than infix, notation to write formulae. For example, where 'C' is the prefix-style implication operator, you often see formulae written using the style on the left, not that on the right:

$$CCpCqrCCpqCpr \equiv (p \to (q \to r)) \to ((p \to q) \to (p \to r)) \tag{S}$$
$$CpCqp \equiv (p \to (q \to p)). \tag{K}$$

I hope you agree that the prefix notation is pretty awkward to read.[1]

So, I wanted a program to convert from prefix to infix notation. This is done by parsing valid prefix strings into ASTs, and then printing those trees in infix format (i.e., by in-order traversal). Rather than use an existing library like Parsec, this was a nice opportunity to implement a simple parser by hand, following the tutorials from Hutton and Meijer (1996, 1998).

1. The examples correspond to the types of the combinators **S** and **K** in combinatory logic.